SR UNIVERSITY

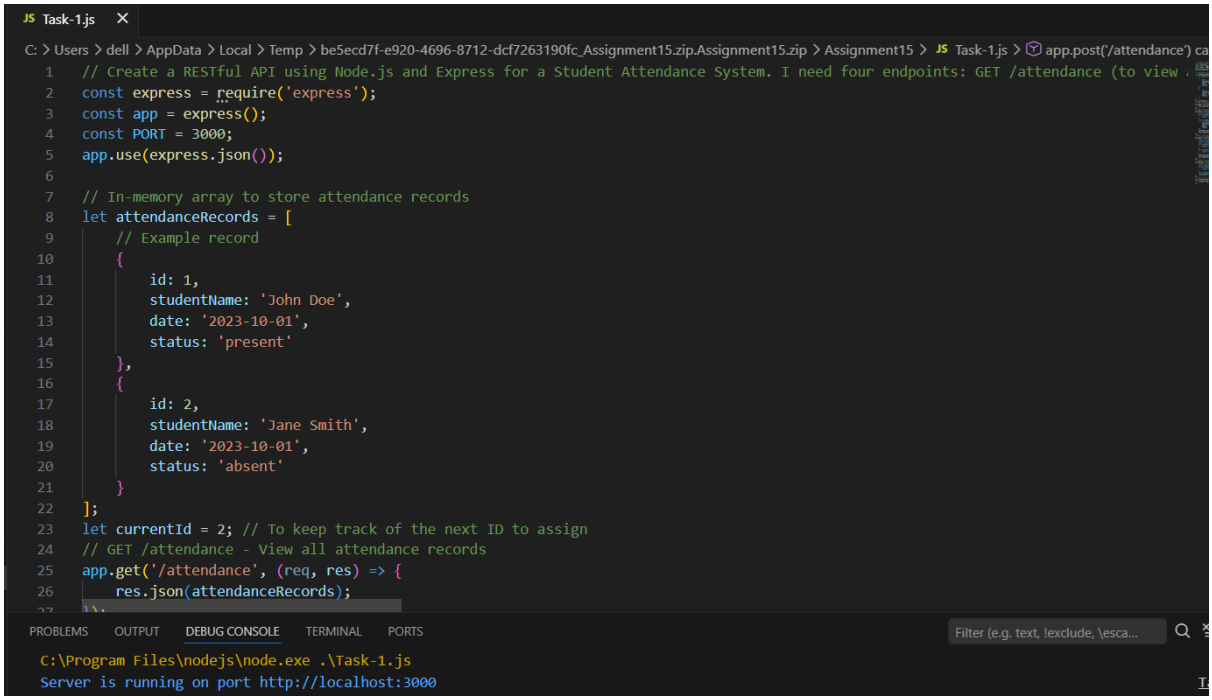
School of Computer Science and Artificial Intelligence

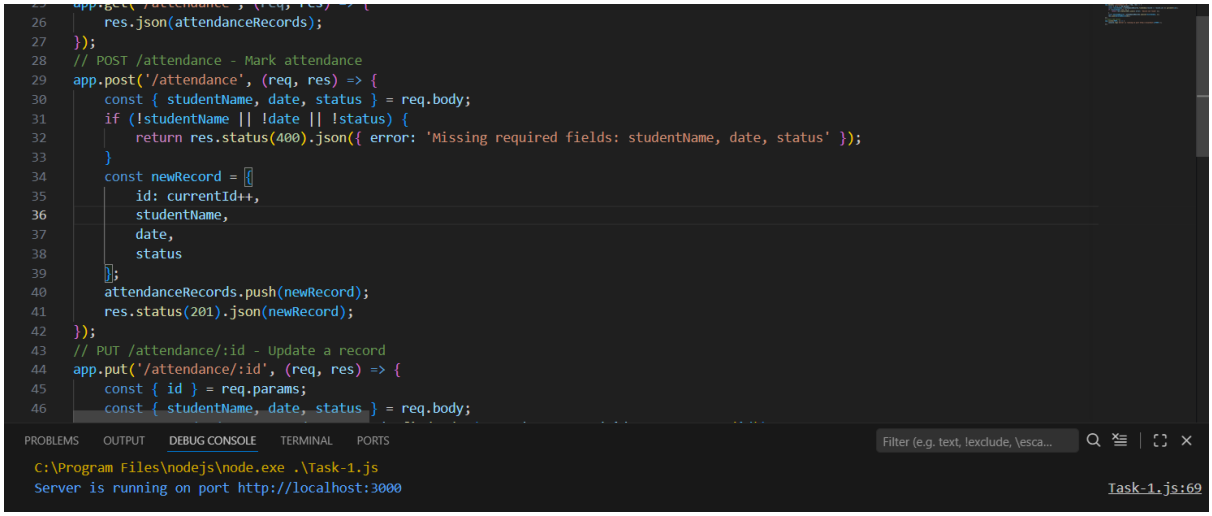
Lab Assignment # 15

Name of Student : Chatharaboina Sagar
Enrollment No. : 2303A51522
Batch No. : 22

Task 1 – Student Attendance API
Create a RESTful API for student attendance tracking.
Instructions:
• Endpoints required:
o GET /attendance → View attendance records
o POST /attendance → Mark attendance
o PUT /attendance/{id} → Update attendance record
o DELETE /attendance/{id} → Remove attendance entry
• Ensure JSON-based request and response handling.
Expected Output:
• API for maintaining and updating attendance records.

Prompt:-
Create a **RESTful API for a Student Attendance System using Node.js and Express** with four endpoints: **GET /attendance** (view all records), **POST /attendance** (add studentName, date, status), **PUT /attendance/:id**
(update record), and **DELETE /attendance/:id** (delete record).
Use a **simple in-memory array** to store attendance data.
Include **basic error handling** for missing fields and invalid IDs.
Return all responses in **JSON format** and add **clear comments in the code** for documentation
Code:-





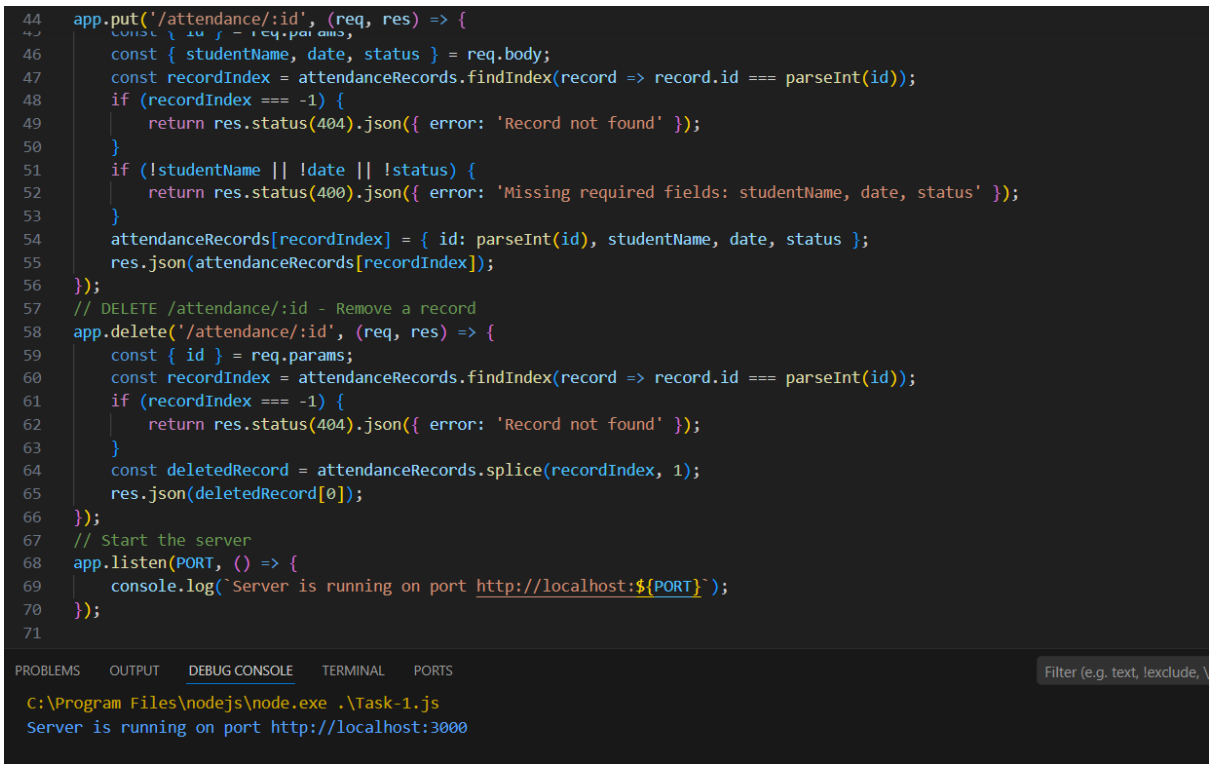
```
    res.json(attendanceRecords);
});
// POST /attendance - Mark attendance
app.post('/attendance', (req, res) => {
    const { studentName, date, status } = req.body;
    if (!studentName || !date || !status) {
        return res.status(400).json({ error: 'Missing required fields: studentName, date, status' });
    }
    const newRecord = {
        id: currentId++,
        studentName,
        date,
        status
    };
    attendanceRecords.push(newRecord);
    res.status(201).json(newRecord);
});
// PUT /attendance/:id - Update a record
app.put('/attendance/:id', (req, res) => {
    const { id } = req.params;
    const { studentName, date, status } = req.body;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Filter (e.g. text, !exclude, \esca...

```
C:\Program Files\nodejs\node.exe .\Task-1.js
Server is running on port http://localhost:3000
```

Task-1.js:69



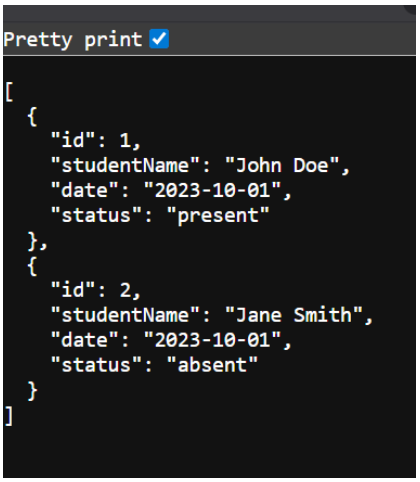
```
app.put('/attendance/:id', (req, res) => {
    const { studentName, date, status } = req.body;
    const recordIndex = attendanceRecords.findIndex(record => record.id === parseInt(id));
    if (recordIndex === -1) {
        return res.status(404).json({ error: 'Record not found' });
    }
    if (!studentName || !date || !status) {
        return res.status(400).json({ error: 'Missing required fields: studentName, date, status' });
    }
    attendanceRecords[recordIndex] = { id: parseInt(id), studentName, date, status };
    res.json(attendanceRecords[recordIndex]);
});
// DELETE /attendance/:id - Remove a record
app.delete('/attendance/:id', (req, res) => {
    const { id } = req.params;
    const recordIndex = attendanceRecords.findIndex(record => record.id === parseInt(id));
    if (recordIndex === -1) {
        return res.status(404).json({ error: 'Record not found' });
    }
    const deletedRecord = attendanceRecords.splice(recordIndex, 1);
    res.json(deletedRecord[0]);
});
// Start the server
app.listen(PORT, () => {
    console.log(`Server is running on port http://localhost:${PORT}`);
});
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Filter (e.g. text, !exclude, \

```
C:\Program Files\nodejs\node.exe .\Task-1.js
Server is running on port http://localhost:3000
```

Output:-



justification:-

This code creates a simple RESTful API to manage student attendance records using **Node.js and Express**. It provides endpoints to view all attendance records, add a new record, update an existing record, and delete a record. The attendance data is stored in a basic **in-memory array**, which makes it suitable for testing and learning purposes. The API also includes **basic error handling** to check if the required fields are provided and to ensure that updates or deletions are performed only on existing records. All responses are returned in **JSON format**, making it easy for front-end applications to interact with the API .

Task 2 – Inventory Management API
Task:
Develop a REST API for warehouse inventory management.
Instructions:
• Endpoints required:
o GET /inventory → List all items
o POST /inventory → Add new item
o PUT /inventory/{id} → Update stock quantity
o DELETE /inventory/{id} → Remove an item
• Implement validation for stock values.
Expected Output:
• API capable of managing inventory stock levels.

Prompt:-
Create a **RESTful API for Warehouse Inventory Management using Node.js and Express** that stores items with **id, name, and stock quantity** in an **in-memory array**.
Include endpoints: **GET /inventory** to list all items and **POST /inventory** to add a new item.
Provide **PUT /inventory/:id** to update the stock quantity of an item.
Add **DELETE /inventory/:id** to remove an item from the inventory.
Include **validation to prevent stock quantity from being negative** and return a **400 error** if invalid, with responses in **JSON format**.
Code:-

C: > Users > dell > AppData > Local > Temp > eef76277-2673-4cf7-8143-a41e36d20602_Assignment15.zip.Assignment15.zip > Assignment15 > JS Task-2.js > app.get('/inventory') callback

```
//create a RESTful API using Node.js and Express for Warehouse Inventory Management that stores items with an ID, name, and stock q
const express = require('express');
const app = express();
const port = 3000;
app.use(express.json());

let inventory = [
    // Sample item for testing
    { id: 1, name: 'Item A', stock: 10 },
    { id: 2, name: 'Item B', stock: 20 },
    { id: 3, name: 'Item C', stock: 30 }
];
let nextId = 4;

// GET /inventory - List all items
app.get('/inventory', (req, res) => {
    res.json(inventory);
});
// POST /inventory - Add new item
app.post('/inventory', (req, res) => {
    const { name, stock } = req.body;
    if (typeof name !== 'string' || typeof stock !== 'number' || stock < 0) {
        return res.status(400).json({ error: 'Invalid input. Name must be a string and stock must be a non-negative number.' });
    }
    const newItem = { id: nextId++, name, stock };
    inventory.push(newItem);
    res.status(201).json(newItem);
}
```

Task-1.js Task-2.js Task-3.js

C: > Users > dell > AppData > Local > Temp > eef76277-2673-4cf7-8143-a41e36d20602_Assignment15.zip.Assignment15.zip > Assignment15 > JS Task-2.js > app.get('/inventory') callback

```
app.post('/inventory', (req, res) => {
}
);
// PUT /inventory/:id - Update stock quantity
app.put('/inventory/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const { stock } = req.body;
    if (typeof stock !== 'number' || stock < 0) {
        return res.status(400).json({ error: 'Invalid input. Stock must be a non-negative number.' });
    }
    const item = inventory.find(i => i.id === id);
    if (!item) {
        return res.status(404).json({ error: 'Item not found.' });
    }
    item.stock = stock;
    res.json(item);
});
// DELETE /inventory/:id - Remove item
app.delete('/inventory/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const index = inventory.findIndex(i => i.id === id);
    if (index === -1) {
        return res.status(404).json({ error: 'Item not found.' });
    }
    const removedItem = inventory.splice(index, 1);
    res.json(removedItem[0]);
});
// Start the server
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

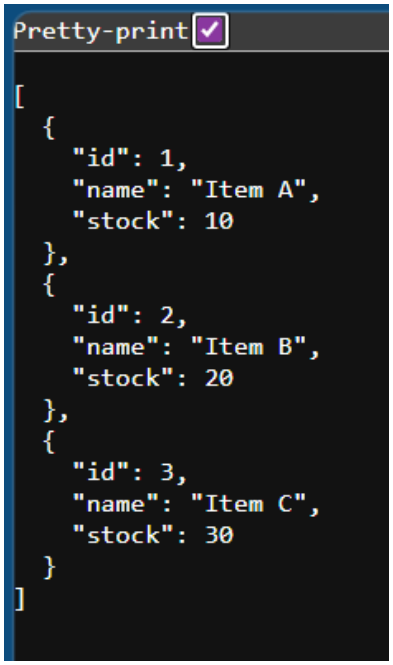
Filter (e.g. text, !exclude, \esca...

```
C:\Program Files\nodejs\node.exe .\Task-2.js
Server is running on port http://localhost:3000
```

```
});
// Start the server
const PORT = 3000;
app.listen(PORT, () => {
    console.log(`Server is running on port http://localhost:${PORT}`);
});
```

Output:-

SR UNIVERSITY



Justification:
This code creates a **RESTful API for Warehouse Inventory Management** using **Node.js and Express**. It allows users to manage inventory items by providing endpoints to **view all items, add new items, update stock quantity, and delete items** from the inventory. The inventory data is stored in a simple **in-memory array**, which is useful for testing and learning purposes. The code also includes **validation checks** to make sure the stock quantity is not negative and returns proper **error messages** if the input is invalid or if the requested item cannot be found.

Task 3 – Hotel Room Booking API

Develop a RESTful API for hotel room booking.
Instructions:
• Endpoints required:
o GET /rooms → View available rooms
o POST /rooms/book → Book a room
o GET /rooms/{id} → View booking details
o DELETE /rooms/{id} → Cancel booking
• Implement error handling for unavailable rooms.
Expected Output:
• API simulating hotel room booking operations.

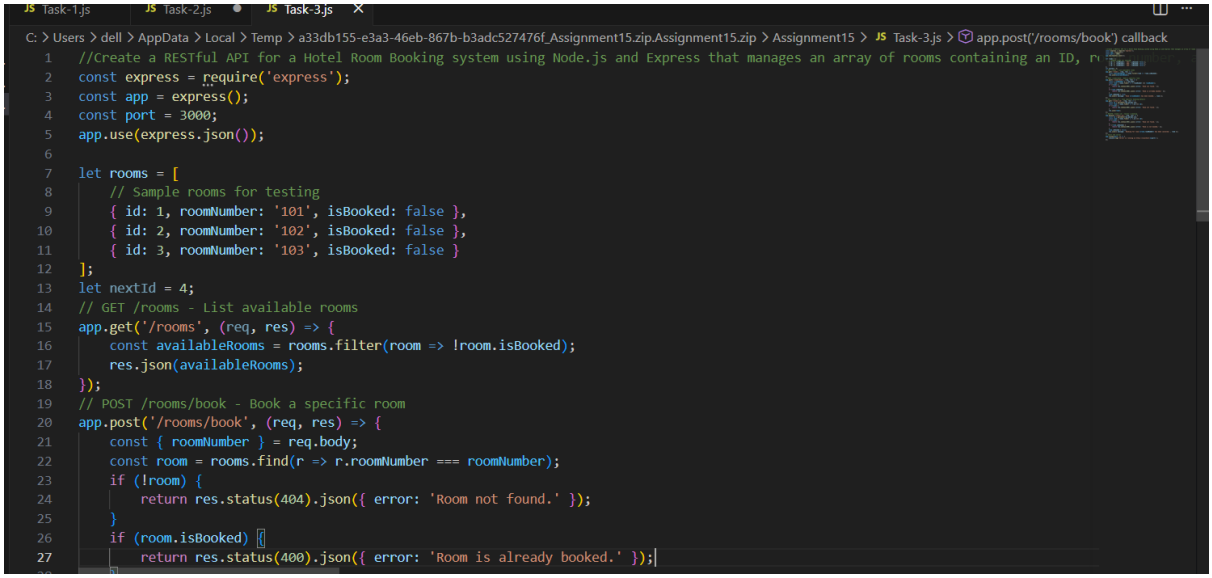
Prompt:-
Create a **RESTful API for a Hotel Room Booking system using Node.js and Express** that manages rooms stored in an **in-memory array** with fields **id, roomNumber, and bookingStatus**.
Include a **GET /rooms** endpoint to return only the **available rooms**.
Add a **POST /rooms/book** endpoint to **book a specific room**.
Provide a **GET /rooms/:id** endpoint to **view booking details of a specific room**.

SR UNIVERSITY

Include a **DELETE /rooms/:id** endpoint to **cancel a room booking**.

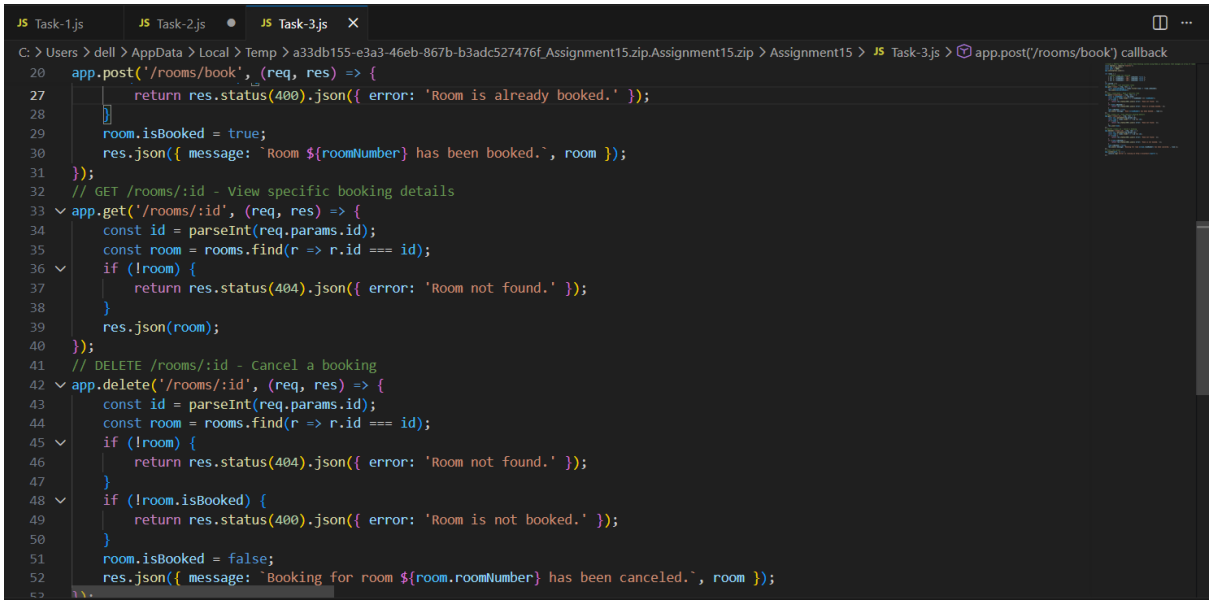
Implement **error handling** to return a message if a user tries to **book an already occupied room or a room that does not exist**, with responses in **JSON format**.

Code:-

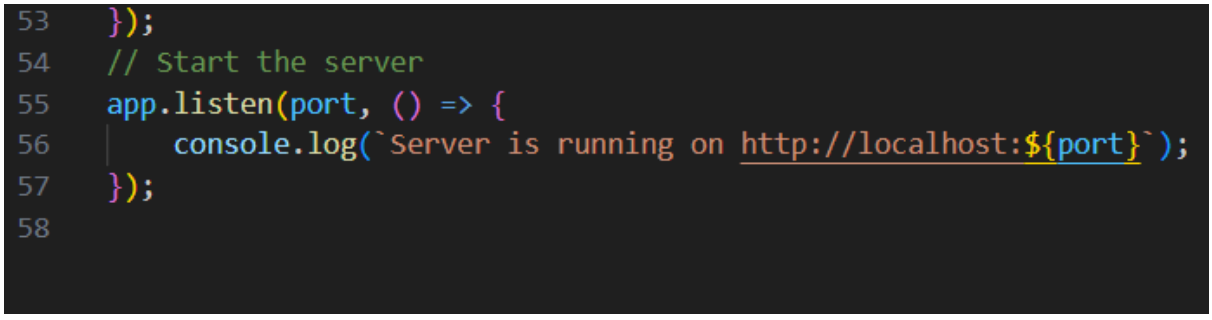


```
//Create a RESTful API for a Hotel Room Booking system using Node.js and Express that manages an array of rooms containing an ID,
const express = require('express');
const app = express();
const port = 3000;
app.use(express.json());

let rooms = [
    // Sample rooms for testing
    { id: 1, roomNumber: '101', isBooked: false },
    { id: 2, roomNumber: '102', isBooked: false },
    { id: 3, roomNumber: '103', isBooked: false }
];
let nextId = 4;
// GET /rooms - List available rooms
app.get('/rooms', (req, res) => {
    const availableRooms = rooms.filter(room => !room.isBooked);
    res.json(availableRooms);
});
// POST /rooms/book - Book a specific room
app.post('/rooms/book', (req, res) => {
    const { roomNumber } = req.body;
    const room = rooms.find(r => r.roomNumber === roomNumber);
    if (!room) {
        return res.status(404).json({ error: 'Room not found.' });
    }
    if (room.isBooked) {
        return res.status(400).json({ error: 'Room is already booked.' });
```

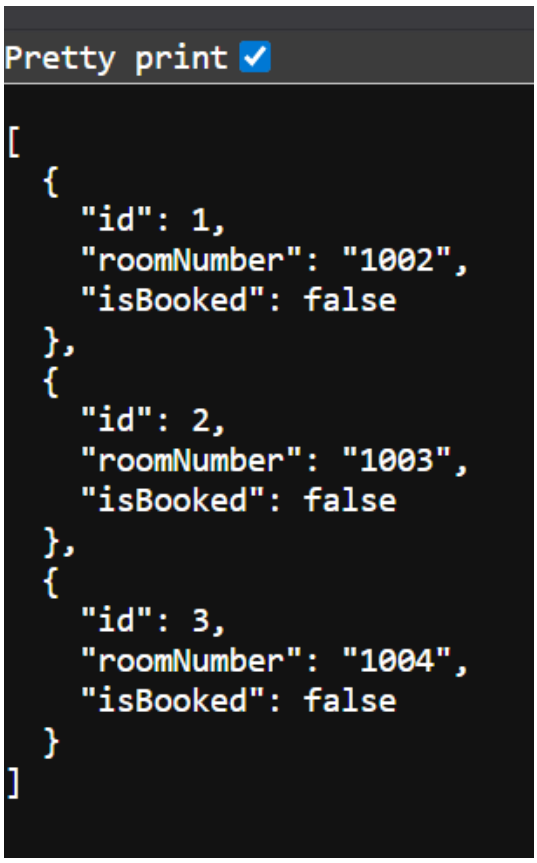


```
app.post('/rooms/book', (req, res) => {
        return res.status(400).json({ error: 'Room is already booked.' });
    }
    room.isBooked = true;
    res.json({ message: `Room ${roomNumber} has been booked.`, room });
});
// GET /rooms/:id - View specific booking details
app.get('/rooms/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const room = rooms.find(r => r.id === id);
    if (!room) {
        return res.status(404).json({ error: 'Room not found.' });
    }
    res.json(room);
});
// DELETE /rooms/:id - Cancel a booking
app.delete('/rooms/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const room = rooms.find(r => r.id === id);
    if (!room) {
        return res.status(404).json({ error: 'Room not found.' });
    }
    if (!room.isBooked) {
        return res.status(400).json({ error: 'Room is not booked.' });
    }
    room.isBooked = false;
    res.json({ message: `Booking for room ${room.roomNumber} has been canceled.`, room });
```



```
});
// Start the server
app.listen(port, () => {
    console.log(`Server is running on http://localhost:${port}`);
});
```

Output:-



Justification:

This code implements a **RESTful API for a Hotel Room Booking system** using **Node.js and Express**. It provides endpoints to **view available rooms, book a specific room, check booking details, and cancel a booking**. The room information is stored in a simple **in-memory array**, which is useful for testing or learning purposes. The API also includes **error handling** to prevent users from booking a room that is already occupied or trying to cancel a booking for a room that does not exist or is not currently booked. All responses are returned in **JSON format** to ensure easy communication with front-end applications

Task 4 – Online Voting API

Create a RESTful API for an online voting system.
Instructions:
• Endpoints required:
o GET /candidates → List candidates
o POST /vote → Cast a vote
o GET /results → View voting results
• Ensure one vote per user using in-memory validation.
Expected Output:
• API simulating a secure online voting process.

Prompt:-
Create a **RESTful API for an Online Voting System using Node.js and Express** that manages a list of **candidates and voting records** using an **in-memory array**.
Include a **GET /candidates** endpoint to display all available candidates.
Add a **POST /vote** endpoint that accepts **userId and candidateId** and records the vote while checking to ensure each user can **vote only once**.
Provide a **GET /results** endpoint to show the **current vote counts for all candidates**.
Implement **validation to prevent duplicate voting** and return a message if a user **tries to vote more than once**, with responses in **JSON format**.

SR UNIVERSITY

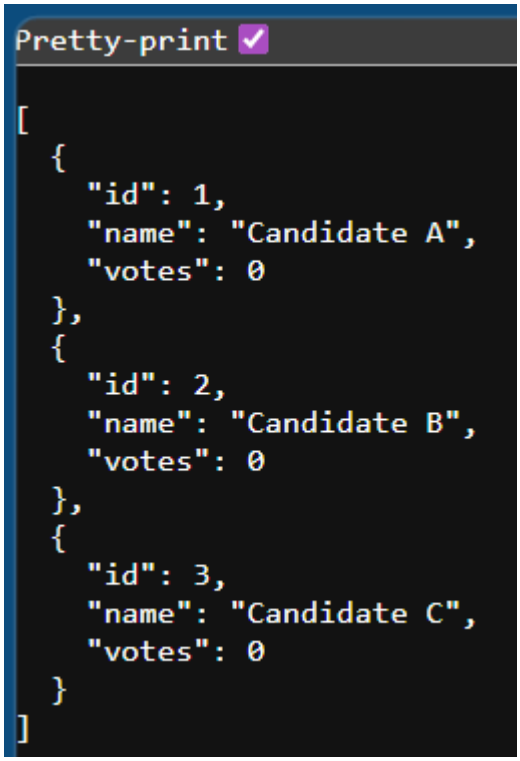
Code:-

Task-1.js Task-2.js Task-3.js Task-4.js

C: > Users > dell > AppData > Local > Temp > 68c5512d-d951-49d0-a19a-f75c400d51fb_Assignment15.zip.Assignment15.zip > Assignment15 > Task-4.js > ...

```
//Develop a RESTful API for an Online Voting System using Node.js and Express that manages a list of candidates and tracks user par
const express = require('express');
const app = express();
const port = 3000;
app.use(express.json());
let candidates = [
    // Sample candidates for testing
    { id: 1, name: 'Candidate A', votes: 0 },
    { id: 2, name: 'Candidate B', votes: 0 },
    { id: 3, name: 'Candidate C', votes: 0 }
];
let votes = []; // To track user votes
// GET /candidates - List all candidates
app.get('/candidates', (req, res) => {
    res.json(candidates);
});
// POST /vote - Cast a vote
app.post('/vote', (req, res) => {
    const { userId, candidateId } = req.body;
    if (!userId || !candidateId) {
        return res.status(400).json({ error: 'User ID and Candidate ID are required.' });
    }
    if (votes.includes(userId)) {
        return res.status(400).json({ error: 'You have already voted.' });
    }
    const candidate = candidates.find(c => c.id === candidateId);
    if (!candidate) {
        return res.status(404).json({ error: 'Candidate not found.' });
    }
    candidate.votes += 1;
    votes.push(userId);
    res.json({ message: `Vote cast for ${candidate.name}.`, candidate });
});
// GET /results - Display current vote totals
app.get('/results', (req, res) => {
    res.json(candidates);
});
// Start the server
app.listen(port, () => {
    console.log(`Server is running on http://localhost:${port}`);
});
```

Output:-



Justification:
This code creates a **RESTful API for an Online Voting System** using **Node.js and Express**. It provides endpoints to **list candidates, cast a vote while ensuring each user votes only once, and display current vote results**. The API also includes **basic error handling to prevent duplicate voting** and returns responses in **JSON format**

Task 5 – Teacher Management API

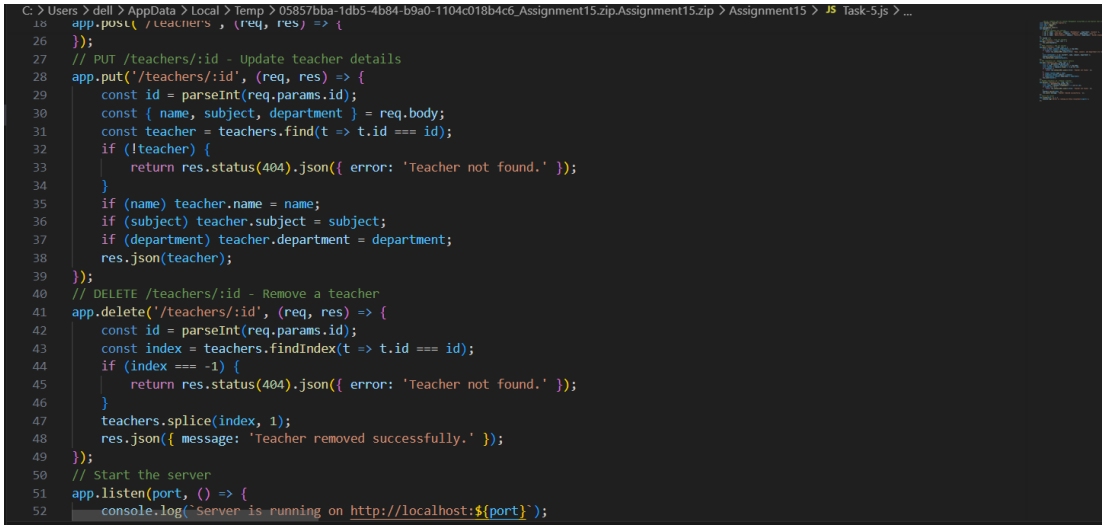
Develop a RESTful API to manage teacher records.
Instructions:
• Endpoints required:
o GET /teachers → List all teachers
o POST /teachers → Add a new teacher
o PUT /teachers/{id} → Update teacher details
o DELETE /teachers/{id} → Remove a teacher
• Ensure proper error handling and JSON responses.
Expected Output:
• Functional API for managing teacher information.

Prompt:-
Create a **RESTful API for Teacher Management using Node.js and Express** to manage teacher records with **id, name, subject, and department**.
Include endpoints **GET /teachers, POST /teachers, PUT /teachers/:id, and DELETE**

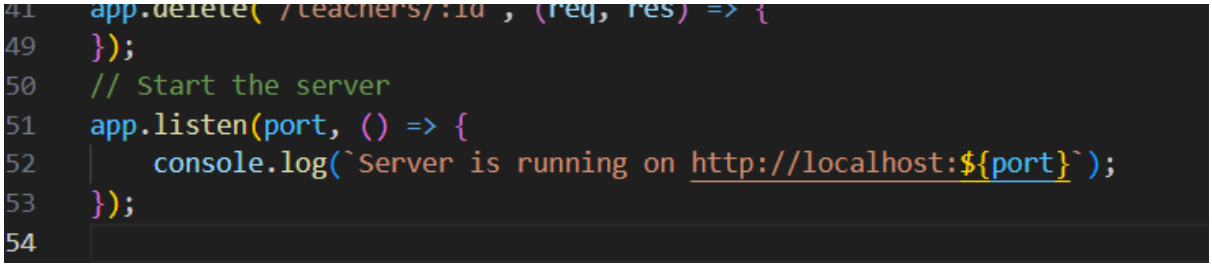
/teachers/:id for full CRUD operations.
Return **JSON responses** and display a **404 error if the requested teacher ID is not found**.

Code:-

```
//Build a RESTful API for Teacher Management using Node.js and Express that allows for full CRUD operations on teacher records cons
const express = require('express');
const app = express();
const port = 3000;
app.use(express.json());
let teachers = [
    // Sample teachers for testing
    { id: 1, name: 'John Doe', subject: 'Mathematics', department: 'Science' },
    { id: 2, name: 'Jane Smith', subject: 'English', department: 'Arts' },
    { id: 3, name: 'Emily Johnson', subject: 'History', department: 'Social Studies' }
];
let nextId = 4;
// GET /teachers - List all teachers
app.get('/teachers', (req, res) => {
    res.json(teachers);
});
// POST /teachers - Add new teacher
app.post('/teachers', (req, res) => {
    const { name, subject, department } = req.body;
    if (!name || !subject || !department) {
        return res.status(400).json({ error: 'Name, subject, and department are required.' });
    }
    const newTeacher = { id: nextId++, name, subject, department };
    teachers.push(newTeacher);
    res.status(201).json(newTeacher);
});
// PUT /teachers/:id - Update teacher details
```

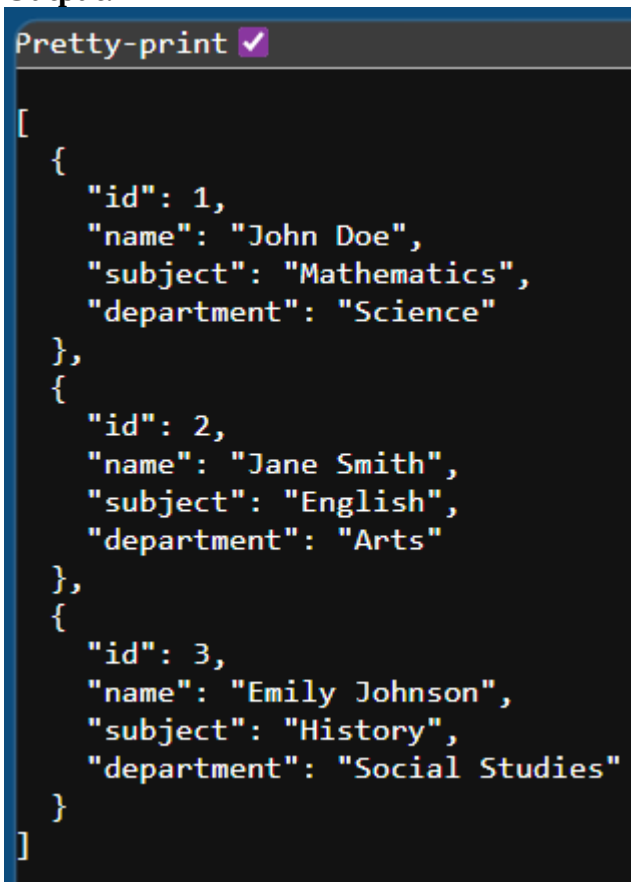


```
// PUT /teachers/:id - Update teacher details
app.put('/teachers/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const { name, subject, department } = req.body;
    const teacher = teachers.find(t => t.id === id);
    if (!teacher) {
        return res.status(404).json({ error: 'Teacher not found.' });
    }
    if (name) teacher.name = name;
    if (subject) teacher.subject = subject;
    if (department) teacher.department = department;
    res.json(teacher);
});
// DELETE /teachers/:id - Remove a teacher
app.delete('/teachers/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const index = teachers.findIndex(t => t.id === id);
    if (index === -1) {
        return res.status(404).json({ error: 'Teacher not found.' });
    }
    teachers.splice(index, 1);
    res.json({ message: 'Teacher removed successfully.' });
});
// Start the server
app.listen(port, () => {
    console.log(`Server is running on http://localhost:${port}`);
```



```
});
// Start the server
app.listen(port, () => {
    console.log(`Server is running on http://localhost:${port}`);
});
```

Output:-



Justification:

This code creates a **RESTful API for Teacher Management** using **Node.js and Express**. It provides endpoints to **view teachers, add new records, update teacher details, and delete teacher records**. The API also includes **basic error handling** to check required fields and returns **JSON responses**, including **404 errors when a teacher ID is not found**.